

Framework Fundamentals: Four Key Layers

1. Fundamentals Layer	2. Data Handling Layer	3. Developer Interface Layer	4. Execution and Abstraction Layer
• Computational graphs - structural basis using DAGs • Enable automatic differentiation and optimization	• Specialized data structures (tensors) • Memory management and device placement	• Programming models (imperative vs symbolic) • Execution models (eager vs graph vs JIT)	• Core operations optimized for diverse hardware • Resource allocation and memory management

Computational Graphs - The Foundation

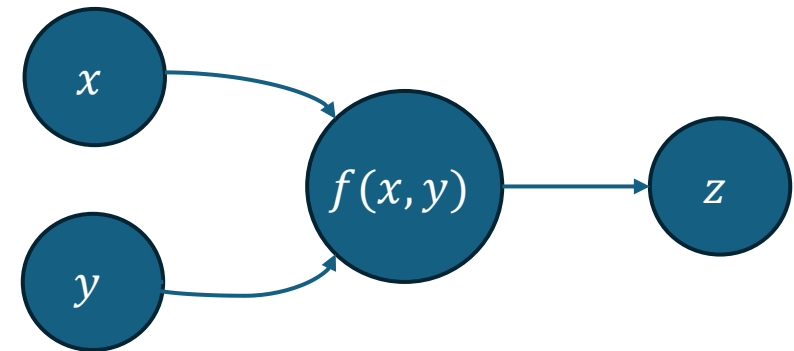
What are Computational Graphs?

- **Directed Acyclic Graphs (DAGs)** representing ML models
- **Nodes represent operations** (matrix multiplication, convolution)
- **Edges represent data flow** between operations

Example: Simple Computation: $z = x \times y$

System Benefits:

- **Automatic differentiation** capabilities
- **Operation scheduling** and optimization
- **Distributed execution** across devices
- **Memory planning** and resource allocation



Static vs Dynamic Computational Graphs

Static Graphs (TensorFlow 1.x style)

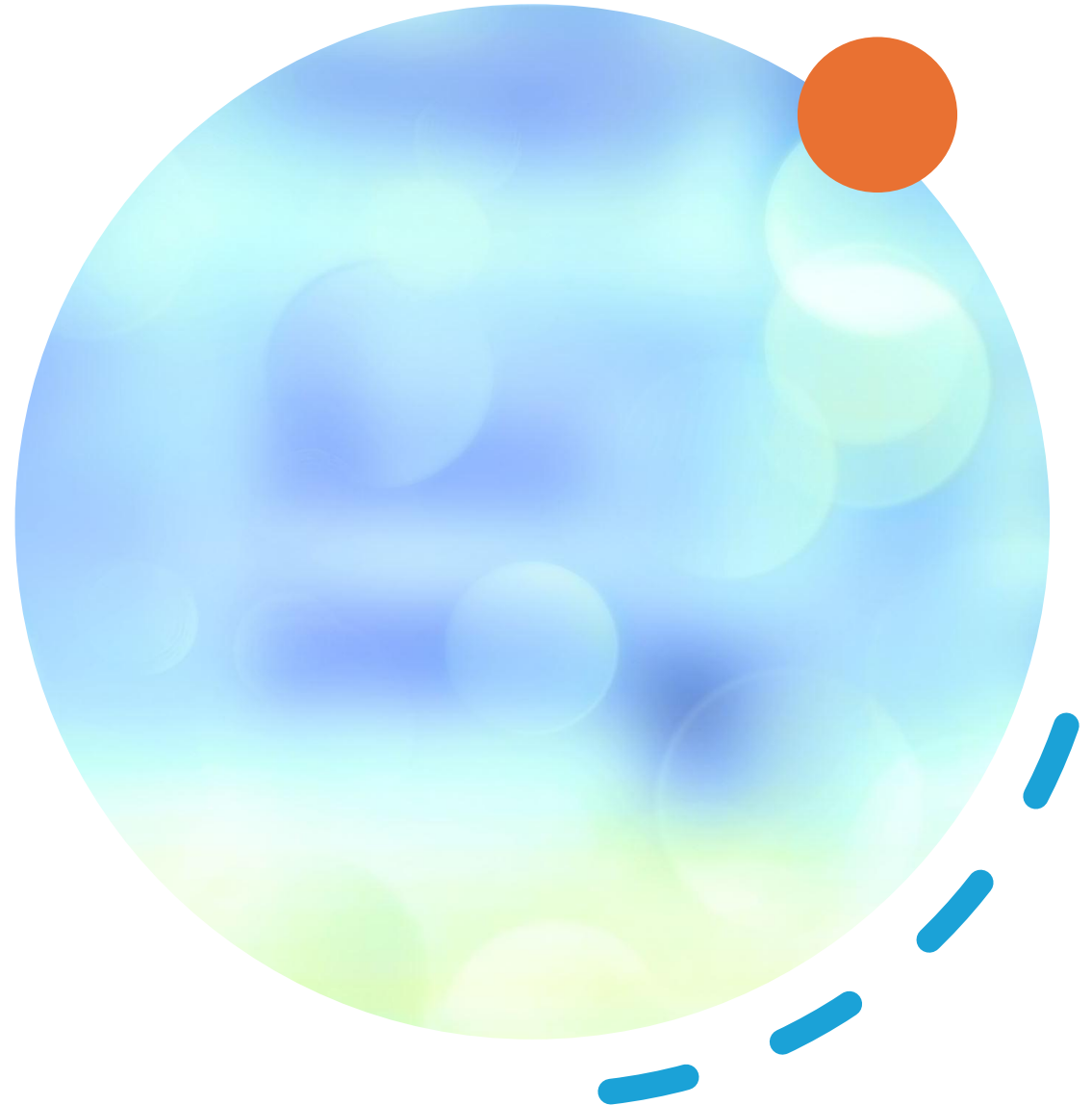
- "Define-then-run" execution model
- **Complete graph specification** before execution
- **Global optimization** opportunities
- **Efficient production deployment**

Dynamic Graphs (PyTorch style)

- "Define-by-run" execution model
- **Graph construction during execution**
- **Immediate feedback** and easy debugging
- **Flexible control flow**

Trade-offs:

- **Static:** Better optimization vs. **Dynamic:** Better flexibility
- **Static:** Production efficiency vs. **Dynamic:** Development ease



Memory Management: Static vs Dynamic

Static Graphs

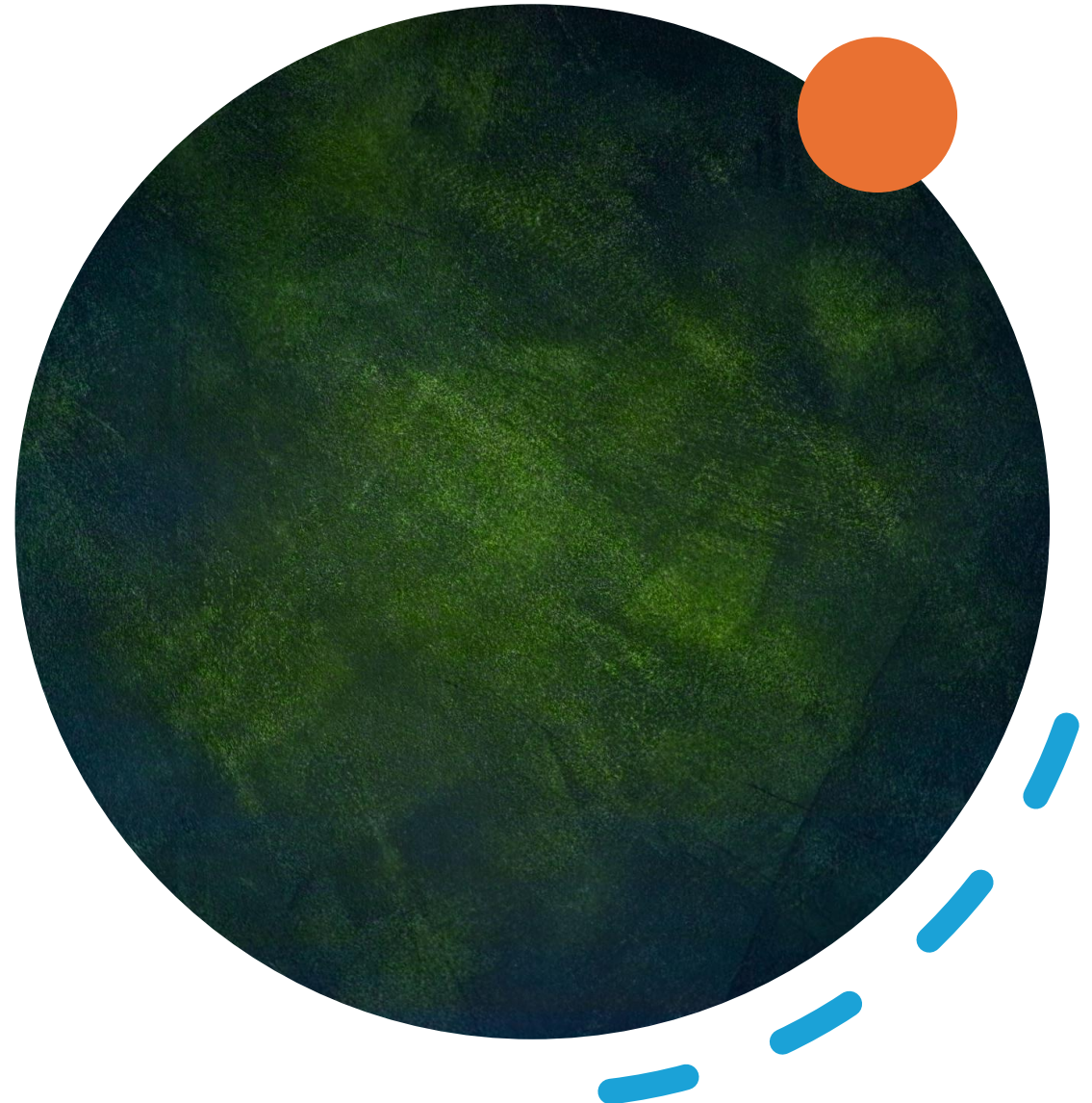
- **Precise memory planning** before execution
- **Optimized allocation** strategies
- **Memory reuse** opportunities
- **Lower memory overhead**

Dynamic Graphs

- **Runtime memory allocation**
- **Flexible but potentially less efficient**
- **Memory fragmentation** risks
- **Higher memory overhead**

System Consequences:

- **Device placement** optimization
- **Memory bandwidth** utilization
- **Resource constraint** handling



Automatic Differentiation: The Engine of Learning



What is Automatic Differentiation?

Systematic computation of derivatives through complex operations

Enables gradient-based optimization for neural network training

Chain rule application across computational graphs

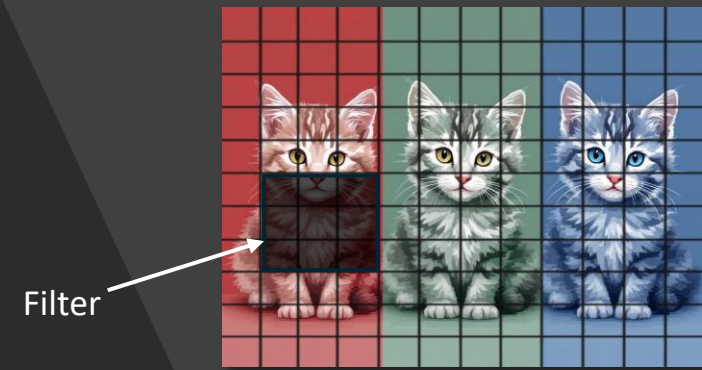
Example: $f(x) = x^2 \times \sin(x)$

AD decomposes into:

1. $a = x^2 \rightarrow$ derivative $2x$
2. $b = \sin(x) \rightarrow$ derivative: $\cos x(x)$
3. Product rule $\rightarrow$ final derivative

```
def f(x):  
    a = x * x           # Square operation  
    b = sin(x)          # Sine operation  
    return a * b        # Product operation
```

CNNs – Fusion Opportunity



Naive Execution Problems:

R: Read from Memory

W: Write to Memory

Input $\rightarrow$ R Conv2D W $\rightarrow$ R BatchNorm W $\rightarrow$ R ReLU W $\rightarrow$ Output

Each operation writes intermediate results to memory

- Separate kernel launches for each operation
- Poor data locality
- Underutilized hardware resources

Optimization Opportunity:

Input $\rightarrow$ R [Conv2D + BatchNorm + ReLU] W $\rightarrow$ Output

- Fused operations eliminate intermediate memory writes and reduce kernel launches.
- It is like taking a direct highway instead of stopping at every traffic light!

Kernel Fusion Techniques

Merging Operations for Efficiency

Common Fusion Patterns:

1. Conv-BatchNorm-Activation

- $conv_output = conv2d(input, weights) + bias$
- $norm_output = batch_norm(conv_output, scale, offset)$
- $final_output = relu(norm_output)$
- Each step writes to memory and reads from memory

After fusion, the above operations become a single (fused) kernel with no intermediate memory writes.

2. Element-wise Operation Fusion

Unfused:

Fused

- $result[i] = relu((x[i] + y[i]) \times z[i])$

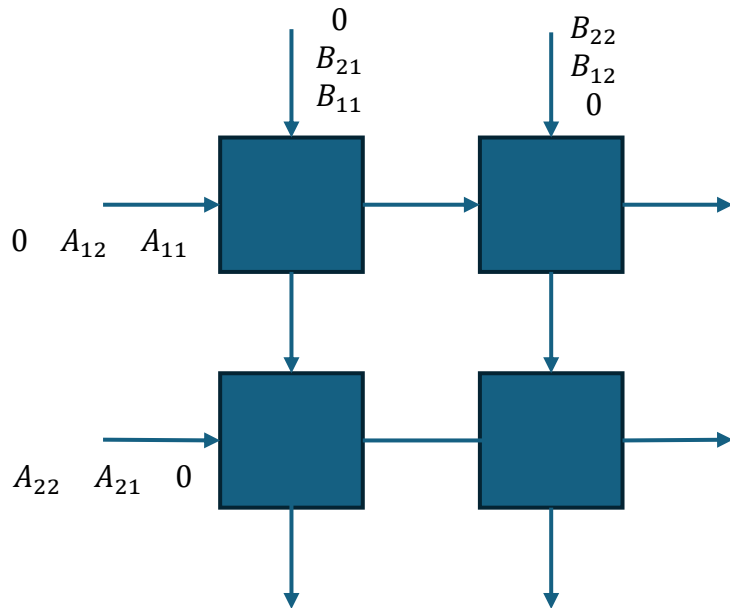
Memory Access Pattern Optimization

Optimizing Data Movement

- **Tiling for Systolic Arrays**
- **Data Layout Transformations**
- **Cache-aware Scheduling**

Tiling for Systolic Arrays

- A **systolic array** is a parallel computing hardware structure of connected processing elements (cells) that rhythmically process and transmit data
- **TPUs** use systolic arrays as their core computational component to efficiently perform the high-throughput matrix multiplication required for AI workloads



Systolic 2x2 multiplication example: A tiles shift to the right and B tiles shift down. Each processing element multiplies its local A and B values and adds to a running sum. After several beats, the correct outputs emerge at the bottom/right without ever stopping the flow.

A "tile" is a pre-defined, fixed-size chunk of a larger matrix.

$$A \times B = C$$

A_{11}	A_{12}	A_{13}	0
A_{21}	A_{22}	A_{23}	0
A_{31}	A_{32}	A_{33}	0
0	0	0	0

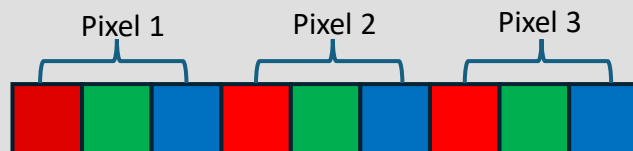
B_{11}	B_{12}	B_{13}	0
B_{21}	B_{22}	B_{23}	0
B_{31}	B_{32}	B_{33}	0
0	0	0	0

Both Matrices A and B are partitioned into fixed-size tiles that match the array's row bandwidth. Zeros pad the boundary, so every tile is the same size and enters on the same rhythm. As each A tile streams to the right and each B tile streams to the bottom, its values are reused across multiple columns of compute cells, letting one load of A and B participate in many multiplications before it exits.

Data Layout Transformations

- N: Number of images in the batch.
- C: Number of Channels per image (e.g., 3 for Red, Green, Blue).
- H: Height of the image in pixels.
- W: Width of the image in pixels.

NHWC



- **Order:** Batch, Height, Width, Channels
- **What it means:** For each pixel, all its color/feature values are stored next to each other.
- **Typical:** TPUs/mobile NPUs; kernels that combine channels at a pixel
- Order in the above image: pixel 1(red)-> pixel 1 (green), pixel 1 (blue), pixel 2(red), pixel 2 (green), pixel 2 (blue), pixel 3(red), pixel 3 (green), pixel 3 (blue)

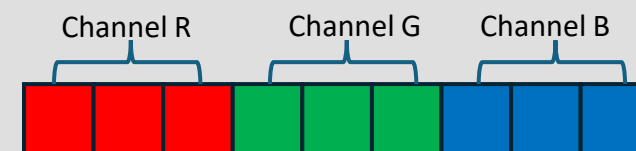
Reorder/transpose axes



Example permutation: [0, 3, 1, 2]
turns NHWC into NCHW

*No data changes—just the order
we store it.*

NCHW



- **Order:** Batch, Channels, Height, Width
- **What it means:** For each channel, all the pixels in the image are stored next to each other.
- **Good when:** You sweep over images one channel at a time.
- **Typical:** Convolution on many GPUs

For large matrix multiplies, we often go beyond NHWC/NCHW and use packed tile layouts that match the GEMM kernel

GEMM stands for General Matrix Multiplication. It's the mathematical name for the core operation that powers deep learning and scientific computing: multiplying two large matrices together.

Cache-aware Scheduling

Cache is a small, fast memory layer that sits between the CPU and main memory to reduce data access latency. It stores recently used (and nearby) data so the processor can reuse it quickly. L1 cache is the smallest and fastest, located closest to the CPU cores and split into instruction/data caches. L2 is larger but a bit slower, often private per core. L3 is much larger and slower than L1/L2, typically shared across cores on a chip, helping reduce trips to main memory by serving common data.

Why it matters:

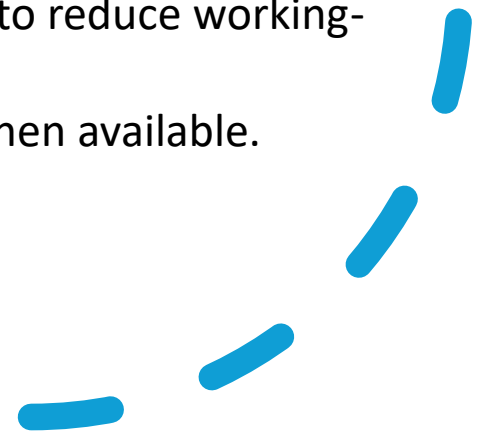
- Minimizes cache misses by keeping hot data in faster caches (L1/L2) longer.

How it works:

- Reorder and group operations to reuse data while it's still in cache.
- Tile/block loops so working sets fit in L1/L2; promote shared data to L3-friendly batches.
- Align access patterns for spatial and temporal locality.

Practical tactics:

- Loop tiling/blocking, fusion, and interchange to reduce working-set size.
- Prefetching and software-managed buffers when available.





Computation Reordering

*Do expensive work less often, and
when you do it, make it count.*

```
# Inefficient: Recompute key-value pairs for each head
for head in attention_heads:
    keys = linear_transform(input, W_k[head])
    values = linear_transform(input, W_v[head])
    attention_output[head] = attention(queries[head], keys, values)

# Optimized: Compute all keys and values once
all_keys = linear_transform(input, W_k_combined)
all_values = linear_transform(input, W_v_combined)
for head in attention_heads:
    attention_output[head] = attention(queries[head],
                                      all_keys[head], all_values[head])
```

Parallelization Opportunities:

Identify operations that can execute concurrently without data dependencies.



Redundant Computation Elimination

Identifying and Removing Unnecessary Work

Common Redundancy Patterns:

1. Duplicate Subexpressions:

```
# Redundant
loss1 = mse(predictions, targets)
loss2 = mse(predictions, targets) # Same computation

# Optimized
shared_loss = mse(predictions, targets)
loss1 = shared_loss
loss2 = shared_loss
```

2. Residual Connection Optimization:

```
# In ResNet blocks, common subexpressions appear
def residual_block(x):
    conv1_out = conv2d(x, weights1)
    conv2_out = conv2d(conv1_out, weights2)
    return x + conv2_out # x is reused
```

3. Constant Folding:

```
# Constants computed at compile time
scaling_factor = 1.0 / 255.0 # Computed once, not per
inference
normalized = input * scaling_factor
```

Identifying and Removing Unnecessary Work



Advanced Graph Transformations

Sophisticated Optimization Techniques

1. Loop Unrolling for RNNs:

```
# Original: Dynamic loop
for t in range(sequence_length):
    hidden[t] = rnn_cell(input[t], hidden[t-1])

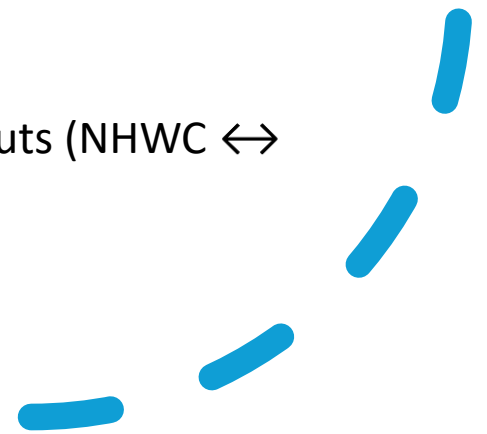
# Optimized: Unrolled for fixed sequence lengths
hidden[0] = rnn_cell(input[0], initial_hidden)
hidden[1] = rnn_cell(input[1], hidden[0])
hidden[2] = rnn_cell(input[2], hidden[1])
# ... enables further optimizations
```

2. Algebraic Simplifications:

```
# Matrix multiplication reordering
#  $A \times (B \times C)$  vs  $(A \times B) \times C$ 
# Choose order based on dimensions to minimize operations
```

3. Memory Layout Transformations:

Automatic conversion between different tensor layouts (NHWC ↔ NCHW) based on hardware preferences



Compiler-based Graph Optimization

Automated Transformation Systems

Pattern Recognition and Rewrite Rules:

```
# Compiler identifies patterns and applies transformations
pattern = "Conv2D → BatchNorm → ReLU"
replacement = "FusedConvBnRelu"

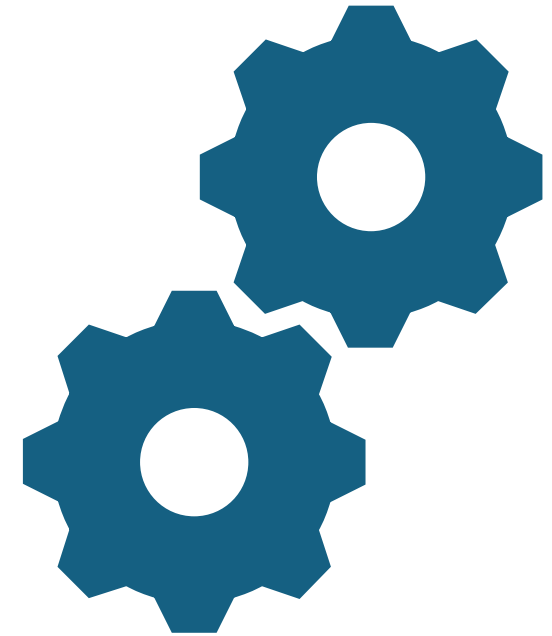
# Systematic application across entire graph
for subgraph in computation_graph:
    if matches_pattern(subgraph, pattern):
        replace_with(subgraph, replacement)
```

Multi-stage Optimization:

1. **High-level transformations:** Operation fusion, redundancy elimination
2. **Hardware-specific optimizations:** Layout transformations, kernel selection
3. **Low-level optimizations:** Instruction scheduling, register allocation

Optimization Verification:

Ensure mathematical equivalence through formal verification or numerical testing.



Graph Optimization Impact

Performance and Efficiency Gains

Bottleneck Mitigation:

- **Memory stalls:** Reduced through better data locality
- **Kernel launch overhead:** Minimized via operation fusion
- **Cache misses:** Improved through access pattern optimization

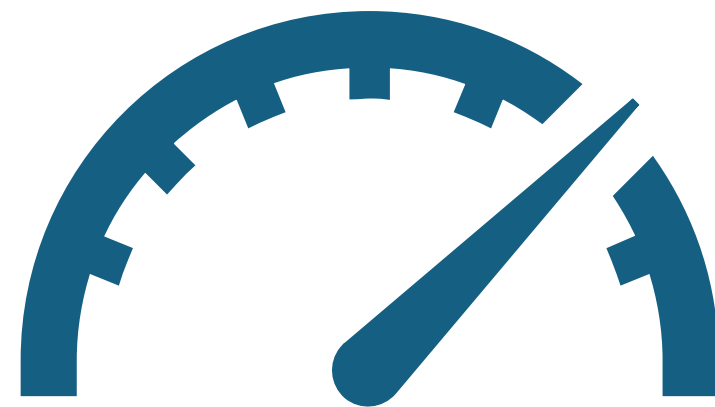
Hardware Utilization:

- **TPUs:** Optimized matrix multiplication tiling
- **GPUs:** Coalesced memory access patterns
- **CPUs:** Vectorization-friendly operations

Real-world Example:

Large Transformer model on edge device:

- **Before optimization:** 2.5 seconds inference, high memory bandwidth
- **After optimization:** 0.8 seconds inference, 60% reduction in memory traffic
- **Enabled real-time operation** on resource-constrained hardware



Data Parallelism Implementation

DDP stands for Distributed Data Parallel, a training wrapper that replicates a model on multiple devices and averages gradients across them via collectives like all-reduce. After synchronization, each replica applies the same optimizer step so all copies stay in lockstep for correct, scalable training.

Steps

1. Deterministic, non-overlapping data sharding per rank
2. Forward + backward per device independently
3. Collective gradient synchronization (all-reduce)
4. Identical parameter updates on all devices

Good practices

- Use a seeded `DistributedSampler` for dataloaders
- Overlap communication and compute via gradient bucketing
- Configure DDP for unused params when models branch (`find_unused_parameters`)
- Gradient clipping before the optimizer step
- Mixed precision: prefer BF16 if available; otherwise FP16 with dynamic loss scaling

Notes on synchronization

- Ring all-reduce minimizes bandwidth and scales with $N-1$ hops
- Ensure optimizer steps are identical across ranks after sync